

Refactoring Report

Participants

- [JIAHAO LI] (Main Developer)
- [JIAHAO LI] (Code Reviewer)

Refactoring Techniques Applied

1. Extract Function

- Extracted `calculate_total_with_tax` from `generate_report`
- Extracted `get_customer_status` from `generate_report`
- These extractions improved code readability and maintainability

2. Rename Variable

- Changed unclear variable names:
 - `y` → `customer_name`
 - `x` → `purchases`
 - `a` → `total_amount`
 - `z` → `purchase`
- Improved code readability and understanding

3. Extract Variable

- Added class-level constants:
 - `priority_threshold = 800`
 - `vip_threshold = 1000`
 - `potential_discount_threshold = 300`
- Made thresholds more maintainable and configurable

4. Simplify Conditional Logic

- Replaced complex nested if-else statements with cleaner logic
- Used list to store customer statuses
- Improved code readability and maintainability

5. Optimize Performance

- Replaced `if name in self.customers.keys()` with `if name in self.customers`
- Used `any()` function for more efficient iteration
- Reduced redundant calculations

Verification of Behavior Preservation

All 17 test cases passed successfully after refactoring, confirming that the observable behavior remains unchanged. The tests covered:

- Customer management (add, update)
- Tax calculation
- Discount eligibility
- Shipping fee calculation
- Report generation
- Various threshold conditions

Benefits and Drawbacks

Benefits

1. Improved Readability

- Clear method names
- Descriptive variable names
- Logical code organization

2. Enhanced Maintainability

- Smaller, focused methods
- Centralized threshold values
- Easier to modify business rules

3. Better Performance

- Optimized iterations
- Reduced redundant calculations
- More efficient data access

4. Easier Testing

- Isolated functionality
- Clear method responsibilities
- Better test coverage

Drawbacks

1. Slightly Increased Code Size

- More methods and variables
- Additional abstraction layers

2. Learning Curve

- New developers need to understand the refactored structure
- More files to navigate

Reflection

The refactoring process was definitely worth the effort. The benefits of improved code quality, maintainability, and performance far outweigh the minor drawbacks. The code is now:

- More professional
- Easier to understand
- More maintainable
- Better performing
- More aligned with Python best practices

The successful test results confirm that the refactoring maintained all existing functionality while significantly improving the code structure.